

树与二叉树

层次关系怎样递归表示、遍历并保持结构不变量

408 · 数据结构 Topic DS-04

母问题：层次关系怎样变成可执行的访问顺序？

树的生成链是

Hierarchy → Recursive Representation → Frontier Traversal → Local Change → Structural Invariant.

节点的子树仍是树，所以递归定义自然产生递归算法；把尚未展开的节点放入栈或队列，就得到显式遍历。访问时机决定先序、中序、后序，取出规则决定层序。Huffman 则把“树”用于编码优化：反复合并最小权值，目标是带权路径长度。

本册定位与 Owner 边界

- **Owns:** 树/森林与二叉树的性质和存储、孩子兄弟转换、递归与迭代遍历、线索化、高度、Huffman 主干。
- **Uses:** DS02 的栈/队列端点契约；DS05 的优先队列接口。
- **Stops:** 堆内部上滤下滤、BST/AVL/B/B+ 查找与旋转、图遍历正确性归各自 Topic。

目录

1 对象与表示	3
2 遍历：访问时机与 frontier	3
3 树、森林与二叉树：孩子兄弟表示是可逆接口	3
3.1 树与二叉树的基本术语必须先固定	3
3.2 遍历序列构造：中序负责切分，另一个序列负责定根	3
3.3 必然会复原的计数不变量	4
3.4 存储表示：选择由操作合同决定	4
3.5 遍历的统一迭代模板与应用	4
3.6 层序的最小深度：首个叶节点即可停止	5
3.7 从遍历序列恢复结构的边界	5
3.8 森林转换与遍历对应	5
3.9 序列化与反序列化：空子树也是信息	5
3.10 后序返回值：让子树摘要替代重复扫描	6
4 线索二叉树：把空指针改造成遍历序列邻接关系	6
4.1 中序线索化的状态机	6

5 Huffman：树是合并结果，不是搜索树	6
5.1 结构不变量与三大关键推论	7
5.2 贪心最优性与唯一性判据	7
6 代码：从节点所有权到访问顺序	7
6.1 节点与递归遍历	7
6.2 显式栈与层序队列	8
6.3 Huffman 合并调用	9
6.4 测试证据	10
7 复杂度与概念边界	11
8 做题控制协议与压缩复原	11

1 对象与表示

二叉树节点至少包含值、左子树、右子树。子树为空是合法状态，不是异常；叶节点的两个孩子都为空。用唯一所有权表示时，父节点拥有子节点，销毁根即可递归释放整棵树。树的结构不变量是：每个节点至多有一个父关系入口，沿孩子指针不会回到祖先，且遍历所得节点数等于可达节点数。

同一棵树的四种读法

对根 1、左子树 2(4,5)、右子树 3，先序为 1,2,4,5,3，中序为 4,2,5,1,3，后序为 4,5,2,3,1，层序为 1,2,3,4,5。树没有变化，改变的是“何时从 frontier 取出并处理根”。

2 遍历：访问时机与 frontier

递归先序在进入节点时处理；中序在左子树返回后处理；后序在两个子树都返回后处理。显式栈保存尚未完成的调用现场：先序压入右子树再压左子树，才能保证左子树先出栈。中序先沿左链压栈，弹出一个节点后转向右子树。层序使用 FIFO 队列，按深度展开；其队列语义来自 DS02，树的覆盖正确性由本册维护。

3 树、森林与二叉树：孩子兄弟表示是可逆接口

一般树的节点可以有任意多个孩子。孩子兄弟表示只保留两个方向：

firstChild = 第一个孩子, nextSibling = 下一个兄弟.

把 firstChild 解释为二叉树左孩子、nextSibling 解释为右孩子，就得到树到二叉树的转换。森林则把各棵树的根按兄弟链连接：第一棵树根是二叉树根，后续树根依次位于右链。

该转换保持层次与兄弟次序，不保持“二叉树左右孩子”的原语义。由指针解释可推出遍历对应：

树的先根遍历 = 对应二叉树的先序遍历, 树的后根遍历 = 对应二叉树的中序遍历.

森林遍历还要沿根的兄弟链完成每棵树，不能只处理第一棵。

3.1 树与二叉树的基本术语必须先固定

树是一个连通、无环的层次关系；根节点没有双亲，其余节点恰有一个双亲。节点的度是孩子数，树的度是所有节点度的最大值；叶节点度为零。深度从根到节点的边数计，树高是最大深度，空树高度究竟取 -1 还是 0 必须和递归公式保持一致。二叉树不是“每层最多两个节点”，而是每个节点有有序的左子树和右子树，空子树也属于结构的一部分。

二叉树的五类形态不能混写：满二叉树每个内部节点都有两个孩子；完全二叉树按层序编号时没有“右孩子存在而左孩子为空”的空洞，最后一层靠左；平衡二叉树是高度差受限的条件；二叉排序树还增加关键字次序；线索二叉树则改变空指针字段的解释。它们共享节点形状，但约束、目标和判定方法不同。

3.2 遍历序列构造：中序负责切分，另一个序列负责定根

只给前序或后序序列通常不能唯一确定二叉树；必须有中序序列提供“左子树元素集合—根—右子树元素集合”的切分。前序与中序构造时，前序当前首元素是根，在中序中找到根的位置后递归切分左右区间；后序与中序构造时，后序当前末元素是根。若关键字允许重复，根在中序中的位置可能不唯一，唯一构造结论失效，题目必须另给节点身份或重复规则。

层序与中序也可以构造：层序剩余区间中最先出现在当前中序区间的节点是该子树根，再按切分后的左右区间过滤后续层序元素。这个过程比前/后序更容易漏掉“过滤不属于当前子树的节点”，所以纸面模拟时应明确维护每个递归子问题的中序边界。

3.3 必然会复原的计数不变量

设二叉树有 n_0 个叶节点、 n_1 个度为 1 的节点、 n_2 个度为 2 的节点。按边数计数：每个非根节点贡献一条入边，因此边数为 $n - 1$ ；按孩子数计数，边数又是 $n_1 + 2n_2$ ，从而

$$n_0 + n_1 + n_2 - 1 = n_1 + 2n_2 \implies n_0 = n_2 + 1.$$

因此真二叉树（没有度为 1 节点）有 $n = 2n_0 - 1$ 个节点；若题目给出叶数或双分支节点数，可以先用这个关系排除不可能的选项。高度为 h （根深度为 0）时，节点数满足 $h + 1 \leq n \leq 2^{h+1} - 1$ ；反过来，给定 n ，最小高度为 $\lceil \log_2(n + 1) \rceil - 1$ ，最大高度为 $n - 1$ 。若教材把空树高度定义为 -1 ，上述递归式应写成 $H(\emptyset) = -1$ 、 $H(T) = 1 + \max(H_L, H_R)$ ；若定义为空树 0，则所有高度整体平移一位，不能混用。

完全二叉树按层序从 1 开始编号时，节点 i 的孩子是 $2i$ 、 $2i + 1$ （越界即不存在），父节点为 $\lfloor i/2 \rfloor$ 。因此最后一个非叶节点编号是 $\lfloor n/2 \rfloor$ ，叶节点恰为 $\lfloor n/2 \rfloor + 1, \dots, n$ ；判断一棵按数组给出的树是否完全，可检查所有非空节点是否已经出现“右孩子而无左孩子”，以及从第一次空槽之后是否仍有非空槽。若采用 0 下标，公式整体改为左孩子 $2i + 1$ 、右孩子 $2i + 2$ ，考场先写下标约定再代入。

3.4 存储表示：选择由操作合同决定

顺序存储只适合完全二叉树或已知形态稳定的树：数组下标直接给出父子关系，但稀疏树会留下大量空槽。二叉链式存储用 left/right 表示真实孩子，任意形态均可表达；含 n 个节点时共有 $2n$ 个孩子字段，其中 $n - 1$ 个为真实边，空字段数为 $n + 1$ ，这正是线索化可利用的空间。一般树可用双亲表示（每节点一个 parent 下标）、孩子链表（每节点挂一条孩子链）或孩子兄弟表示；前者便于向上走，后两者便于枚举孩子。选型时分别问“是否频繁找父亲”“是否频繁插入/删除孩子”“是否需要保留兄弟顺序”，不要因为都是树就默认为同一种节点结构。

孩子兄弟转换的复杂度为 $O(n)$ ，逆转换同样为 $O(n)$ 。转换后右指针是兄弟而非普通孩子，故在纸面递归中必须使用两套语义：沿左指针进入孩子，处理完一棵子树后沿右指针进入下一个兄弟。把右链误当成孩子链会导致节点数重复或树高错误。

3.5 遍历的统一迭代模板与应用

迭代遍历的共同状态是“已发现但尚未完成的 frontier”。先序栈模板为：弹出节点、访问、先压右再压左；中序模板为“沿左链全部压栈，弹出并访问，再令当前指向右子树”；后序模板可用双栈（第一栈根右左出序，第二栈再逆序）或单栈加 lastVisited 记录最近完成的右子树。所有模板都应证明两点：每个真实孩子只被推进一次；空指针不会入栈。于是时间为 $O(n)$ ，额外空间分别受高度 h 、宽度 w 或两栈节点数约束。

层序遍历用队列逐层展开。若要输出层号，在队首记录 level 或在每轮固定当前队列长度；若要求树宽，取各轮最大队列长度；若要求根到节点的路径，队列元素同时保存 parent 或在入队时写 parent 数组。树没有环，通常不需要 visited；一旦节点结构可能共享子树或退化为图，就必须补 visited，否则会重复展开。后序特别适合释放整棵树、计算高度/子树大小、验证平衡和自底向上合并结果；中序在有序树上产生关键字非降序，但“中序有序”只是在 BST 约束已成立时才能作为验证依据。

3.6 层序的最小深度：首个叶节点即可停止

若目标是根到最近叶子的边数，层序遍历按深度递增出队；第一次取到左右孩子均为空的节点时，其层号就是最小深度。空树的返回值必须先约定（常见为 0），根为叶时深度为 1 或 0 取决于题目按节点数还是边数计，不能混写。与求最大深度的 DFS 相比，最小深度的 BFS 可以在首个叶节点停止，但停止前仍要保证本层节点按同一层号处理。

队列元素可保存 $(node, depth)$ ，或用“本轮队列长度 + 层号”推进。每个节点最多入队一次，时间为 $O(n)$ ，最坏空间为树的最大宽度 $O(w)$ ；若树退化为链，BFS 仍可能只保存常数个节点，而递归 DFS 的栈达到 $O(h)$ 。这个应用说明遍历策略由目标合同决定：最短层数偏向 BFS，子树摘要偏向后序 DFS。

3.7 从遍历序列恢复结构的边界

前序 + 中序、后序 + 中序在节点身份唯一时可唯一恢复；递归过程的核心不是口诀，而是维护每个子问题的中序闭区间 $[L, R]$ 与另一序列的根位置。先序根在区间首端，后序根在区间末端；切分后左子树节点数决定另一序列的偏移。层序 + 中序也可恢复，但每次递归必须过滤出属于当前区间的层序节点，不能直接按连续下标切片。

前序 + 后序通常不能唯一恢复，因为只有一个孩子的节点可以把孩子放在左边或右边而不改变两种序列；若额外保证每个内部节点恰有两个孩子（满二叉树），才可由前序下一个节点在后序中的位置切分出左子树，进而唯一恢复。遇到重复关键字时，即使序列组合形式满足上述条件，也只能恢复结构类而非带身份节点，除非题目给出稳定编号。

3.8 森林转换与遍历对应

树转二叉树使用“第一个孩子—下一个兄弟”编码：左链向下表示孩子，右链向右表示兄弟。树的先根遍历对应二叉树先序；树的后根遍历对应二叉树中序。森林转换时，所有树根串成兄弟链，所以对应二叉树根的右子树不再表示同一棵树的普通孩子，而表示下一棵树。恢复时必须按相同语义逆解释，不能把转换后的右链直接当作原树的孩子链。

3.9 序列化与反序列化：空子树也是信息

把树写成字符串或记录流时，目标不是“输出一次遍历”，而是建立可逆编码。仅有先序值序列通常不能区分“只有左孩子”和“只有右孩子”；若在先序流中为每个空子树写入统一标记（如 #），解码器每读一个普通值就递归构造左右子树，每读一个空标记就返回空指针，于是消费次序与结构生成次序一一对应。含 n 个真实节点的二叉树恰有 $n + 1$ 个空孩子位置，因此完整流有 $2n + 1$ 个记号，这也是检查截断、冗余输入和非法语法的计数证据。

层序编码则让队列中的父节点依次消费两个孩子记号；空记号仍不可在未约定语法时随意删除。可以裁去末尾连续空标记，但编码器与解码器必须共同约定“缺失的尾部孩子均为空”，同时拒绝父节点已经为空却又出现后代值的非法流。重复 key 不影响这种带空标记编码，因为结构由位置语法确定；它只会破坏依赖“在中序中定位唯一根”的重建方法。

遍历序列不等于结构编码

前序、后序或层序只记录真实节点的访问顺序；序列化还必须保存足以恢复空边界的信息。可逆性证明应回答两问：每个编码记号由哪个节点/空槽生成？解码时何时唯一知道一个子树已经结束？

3.10 后序返回值：让子树摘要替代重复扫描

许多树题表面不同，实际都让后序递归返回“父节点继续决策所需的最小摘要”。求直径时，左右子树返回高度，当前节点用 $h_L + h_R$ 更新全局最优，再向上返回 $1 + \max(h_L, h_R)$ ；若每个节点都另行扫描子树高度，会把链状树退化为 $O(n^2)$ 。求两个节点的最近公共祖先时，空节点返回空，命中目标返回自身；左右返回都非空则当前根是最低汇合点，只有一侧非空就把该证据向上传递。

原地展开为先序右链时，朴素做法每次把左子树接到右侧后再扫描其最右端，最坏同样为 $O(n^2)$ 。可选择后序返回展开后的尾节点，或按“右—左—根”的逆先序维护上一个已处理节点并重写 `right`，使每个节点只参与常数连接。共同方法是先写清递归合同：**返回值描述哪棵子树的什么摘要，副作用更新什么全局量，空树返回什么单位元。**

4 线索二叉树：把空指针改造成遍历序列邻接关系

含 n 个节点的二叉链表有 $2n$ 个孩子指针，而非空边只有 $n - 1$ 条，因此有 $n + 1$ 个空指针。线索化利用这些空槽保存某种遍历次序下的前驱或后继。因为字段可能表示真实孩子或线索，必须增加标志：

$$ltag = 0 \Rightarrow lchild \text{ 是左孩子}, \quad ltag = 1 \Rightarrow lchild \text{ 是前驱线索},$$

右侧同理。忽略 `tag` 沿线索递归，会把序列关系误当结构边，造成重复访问甚至循环。

4.1 中序线索化的状态机

维护 `pre` 指向刚刚访问的节点，在访问当前节点 p 时：

1. 若 p 的左孩子为空，令 $p.lchild = pre$ 且 $p.ltag = 1$ ；
2. 若 pre 存在且其右孩子为空，令 $pre.rchild = p$ 且 $pre.rtag = 1$ ；
3. 更新 $pre = p$ ，再处理右子树。

这不是任意补指针，而是在同步构造中序序列相邻节点的关系。线索化完成后找中序后继：若 $rtag = 1$ ，线索直接给出后继；若 $rtag = 0$ ，后继是右子树中沿真实左孩子到达的最左节点。

带头结点的中序线索树 若引入头结点 (Header)，其指针约定为：

- 头结点的 `lchild` 指向二叉树根节点，`rchild` 指向自身（或中序最后一个节点）；
- 中序第一个节点的 `lchild` 与最后一个节点的 `rchild` 均指回头结点。

这把“无前驱/无后继”统一编码为环形边界，遍历时无需递归栈与显式判断空指针。

线索树边界

线索只服务于指定遍历次序；中序前驱/后继不能直接当作先序关系。某些先序、后序邻接仍可能需要父指针或重新定位，不能把“有线索”理解成所有遍历操作都严格 $O(1)$ 。

5 Huffman：树是合并结果，不是搜索树

给定符号权重集合 $\{w_1, \dots, w_k\}$ ，带权路径长度定义为 $WPL = \sum_{i=1}^k w_i d_i$ 。

5.1 结构不变量与三大关键推论

1. **严格二叉树（正则二叉树）**：树中只有度为 0（叶节点，共 k 个）与度为 2（内部合并节点，共 $k-1$ 个）的节点，**不存在度为 1 的节点**，总节点数恒为 $2k-1$ ；
2. **叶节点放置字符**：字符必须且只能放置在叶节点，内部节点仅记录合并权值；这保证了前缀码性质（任一字符编码绝不是另一字符的前缀）；
3. **等价计算公式**：全树各内部节点的合并权值总和，严格等于所有叶节点的带权路径长度总和：

$$\text{WPL} = \sum_{\text{leaves}} w_i d_i = \sum_{\text{internal}} \text{weight}(\text{node})$$

5.2 贪心最优性与唯一性判据

- **贪心选择性质**：最小的两个权值必然位于树的最深层且互为兄弟节点，合并后转化为规模更小的同型子问题；
- **编码不唯一性**：左右子树约定（左 0 右 1 还是左 1 右 0）以及并加权值的合并次序可能生成不同形态的树和不同码字，但 **** 最小 WPL 恒定唯一 ****。

译码时从根出发，读到 0/1 就沿对应边走；到达叶节点便输出该字符并回到根。若输入位串在中途停在内部节点，或沿不存在的边行走，编码串非法。判定一组编码是否满足前缀性，可以把每个编码逐位插入一棵二叉 **trie**：插入过程中遇到已有叶节点，或插入结束时节点已有孩子，均说明某个编码是另一个编码的前缀。

前缀码的即时译码性来自“任一码字不是另一码字前缀”：读到叶即可唯一决定字符，不必回看。若题目给出码字集合，可用 **trie** 插入验证，也可按码长排序后做前缀比较；后者在码字很多时要注意字符集和位串长度。**Huffman** 码通常不是固定长度码，压缩收益应与固定长度 $\lceil \log_2 k \rceil$ 的基线比较，而不是只看树高；若权值极不均衡，最长码字可能接近 $k-1$ ，但 **WPL** 仍由频率加权。

6 代码：从节点所有权到访问顺序

完整实现位于 `code/ds04_tree_binary.hpp`，测试位于 `code/ds04_tree_binary_test.cpp`；代码块直接引用真实源文件。

6.1 节点与递归遍历

```

1
2 struct Node {
3     int value;
4     std::unique_ptr<Node> left;
5     std::unique_ptr<Node> right;
6 };
7
8 inline std::unique_ptr<Node> leaf(int value) {
9     return std::make_unique<Node>(Node{value, nullptr, nullptr});
10 }
11
12 inline void preorder_recursive(const Node* root, std::vector<int>& output) {
13     if (root == nullptr) {

```

```

14     return;
15 }
16 output.push_back(root->value);
17 preorder_recursive(root->left.get(), output);
18 preorder_recursive(root->right.get(), output);
19 }
20
21 inline void inorder_recursive(const Node* root, std::vector<int>& output) {
22     if (root == nullptr) {
23         return;
24     }
25     inorder_recursive(root->left.get(), output);
26     output.push_back(root->value);
27     inorder_recursive(root->right.get(), output);
28 }
29
30 inline void postorder_recursive(const Node* root, std::vector<int>& output) {
31     if (root == nullptr) {
32         return;

```

代码清单 1: 唯一所有权节点与递归先中后序

递归基是空指针立即返回；每个非空节点只处理一次，子树调用返回后再按访问时机写入输出。

6.2 显式栈与层序队列

```

1  postorder_recursive(root->left.get(), output);
2  postorder_recursive(root->right.get(), output);
3  output.push_back(root->value);
4  }
5
6  inline std::vector<int> preorder_iterative(const Node* root) {
7      std::vector<int> output;
8      if (root == nullptr) {
9          return output;
10     }
11     std::stack<const Node*> pending;
12     pending.push(root);
13     while (!pending.empty()) {
14         const Node* current = pending.top();
15         pending.pop();
16         output.push_back(current->value);
17         if (current->right != nullptr) {
18             pending.push(current->right.get());
19         }
20         if (current->left != nullptr) {
21             pending.push(current->left.get());
22         }
23     }
24     return output;
25 }
26

```



```

27 inline std::vector<int> inorder_iterative(const Node* root) {
28     std::vector<int> output;
29     std::stack<const Node*> pending;
30     const Node* current = root;
31     while (current != nullptr || !pending.empty()) {
32         while (current != nullptr) {
33             pending.push(current);
34             current = current->left.get();
35         }
36         current = pending.top();
37         pending.pop();
38         output.push_back(current->value);
39         current = current->right.get();
40     }
41     return output;
42 }
43
44 inline std::vector<int> level_order(const Node* root) {
45     std::vector<int> output;
46     if (root == nullptr) {
47         return output;
48     }
49     std::queue<const Node*> pending;
50     pending.push(root);
51     while (!pending.empty()) {
52         const Node* current = pending.front();
53         pending.pop();
54         output.push_back(current->value);
55         if (current->left != nullptr) {

```

代码清单 2: 迭代遍历与层序 frontier

显式结构保存的不是“所有节点”，而是尚未展开的边界；栈的压入顺序和队列的端点顺序直接生成输出次序。

6.3 Huffman 合并调用

```

1     return 0;
2 }
3 return 1 + std::max(height(root->left.get()), height(root->right.get()));
4 }
5
6 inline int huffman_weighted_path_length(const std::vector<int>& weights) {
7     if (weights.size() < 2) {
8         return 0;
9     }
10    std::priority_queue<int, std::vector<int>, std::greater<int>> pending(
11        weights.begin(), weights.end());
12    int total = 0;
13    while (pending.size() > 1) {
14        const int first = pending.top();
15        pending.pop();

```

```

16     const int second = pending.top();
17     pending.pop();
18     const int merged = first + second;
19     total += merged;
20     pending.push(merged);

```

代码清单 3: 最小权值合并与累计代价

6.4 测试证据

```

1  std::unique_ptr<Node> sample_tree() {
2      auto root = leaf(1);
3      root->left = leaf(2);
4      root->right = leaf(3);
5      root->left->left = leaf(4);
6      root->left->right = leaf(5);
7      return root;
8  }
9
10 void test_empty_and_singleton() {
11     std::vector<int> output;
12     preorder_recursive(nullptr, output);
13     assert(output.empty());
14     assert(level_order(nullptr).empty());
15     auto root = leaf(9);
16     assert(height(root.get()) == 1);
17     assert((preorder_iterative(root.get()) == std::vector<int>{9}));
18     assert((inorder_iterative(root.get()) == std::vector<int>{9}));
19 }
20
21 void test_traversal_orders() {
22     auto root = sample_tree();
23     std::vector<int> preorder;
24     std::vector<int> inorder;
25     std::vector<int> postorder;
26     preorder_recursive(root.get(), preorder);
27     inorder_recursive(root.get(), inorder);
28     postorder_recursive(root.get(), postorder);
29     assert((preorder == std::vector<int>{1, 2, 4, 5, 3}));
30     assert((preorder_iterative(root.get()) == preorder));
31     assert((inorder == std::vector<int>{4, 2, 5, 1, 3}));
32     assert((inorder_iterative(root.get()) == inorder));
33     assert((postorder == std::vector<int>{4, 5, 2, 3, 1}));
34     assert((level_order(root.get()) == std::vector<int>{1, 2, 3, 4, 5}));
35     assert(height(root.get()) == 3);
36 }

```

代码清单 4: 空树、遍历覆盖与 Huffman 边界测试

```

1 clang++ -std=c++17 -Wall -Wextra -Werror -pedantic \
2     code/ds04_tree_binary_test.cpp -o /tmp/ds04_test

```

3 /tmp/ds04_test

7 复杂度与概念边界

操作	递归/迭代遍历	额外空间	前提	边界
遍历 n 节点	$O(n)$	$O(h)$ 或 $O(w)$	每节点一次	空树返回空序列
高度	$O(n)$	$O(h)$	子树高度定义固定	空树高度约定
Huffman 合并	$O(k \log k)$	$O(k)$	最小优先队列	$k < 2$ 代价为 0

概念 A	≠ 概念 B	判据	错误后果
树节点所有权	≠ 遍历顺序	生命周期 vs 访问时机	泄漏或次序错
层序遍历	≠ BFS 图遍历	树无环且无需一般 visited	把图结论硬套树
Huffman 树	≠ BST	最小带权路径 vs 有序查找	混淆目标

8 做题控制协议与压缩复原

先画根、子树范围和空孩子；再选访问时机或 frontier；每次节点只入队/入栈一次并验证覆盖；若题目问 Huffman，改写为“合并两个最小权值并累计”。忘掉口诀后复原五问：节点拥有谁？未展开边界在哪里？当前节点何时处理？空子树如何结束？目标是访问次序还是编码代价？

全科交接

DS04 从 DS02 接收 LIFO/FIFO frontier，从 DS05 接收最小优先队列接口；向 DS-B01 提供树遍历实例，向 DS09 提供“树结构可承载有序索引”的前置直觉，但不拥有 BST/AVL 机制。

旧总册关于递归层次、遍历访问时机与 Huffman 合并的内容已吸收；树/森林转换、线索化、遍历序列重建、带空标记序列化与后序摘要已按各自边界补入本册。更复杂的压缩编码和持久化格式仍只作 Extension，不能覆盖本册主干。